International Symposium on Academic Makerspaces — ISAM 2026

The MakerLAB Assistant: AI to Help Operate, Fix, and Build in Makerspaces

Isaac Steinberg[1], Niti Parikh[2], and Miguel Ramirez Peraza[3]

[1]Isaac Steinberg; MBA '26, Johnson Cornell Tech; ies22@cornell.edu

[2]Niti Parikh; Director, Learning Spaces & MakerLABs, Cornell Tech; ntp27@cornell.edu

[3]Miguel Ramirez Peraza; MakerLAB Intern, Cornell Tech; ramirezperazamiguel@gmail.com

***Abstract*—**Academic makerspaces accumulate substantial operational knowledge: machine documentation, setup procedures, safety rules, materials, inventory, and fabrication know-how. That knowledge is scattered across shared drives, URLs, videos, institutional memory, staff, and repair vendors rather than centralized, and it is mostly in English. At a lab like the Cornell Tech MakerLAB—seven-day access, lean staff, students at every skill level—access to this knowledge ends up unevenly distributed, and the learning curve is steep: students must ideate, find the right hardware, recall procedures, and run complex multi-machine setups. We demonstrate the **MakerLAB Assistant**, a conversational AI layered over a structured operational database (hosted in Notion) within the *MakerLAB Tools* app. It gives students instant, anytime access to the inventory and the lab's specifications in plain language and many languages: find the right tool, follow manual-grounded setup and troubleshooting, log a maintenance ticket, and scope multi-machine projects. The same database is exposed as a live Model Context Protocol (MCP) server, so attendees can query the lab from an LLM client such as Claude or ChatGPT. The demonstration is interactive: visitors browse the catalog, scope a project, and troubleshoot an example issue on a 3D printer. The app is already deployed at the Cornell Tech MakerLAB over an initial catalogue of 100 machines, and we frame this as an ongoing R&D initiative whose long-term goal is a shared operational framework across Cornell's campuses while preserving each lab's local identity.

1. Background and Motivation

The project began on the floor of the Cornell Tech MakerLAB. An intern (a co-author) built the first inventory app in Streamlit to search the lab's hardware—work that organized the data and framed the problem. A second co-author, volunteering as a "SuperMaker," kept hitting the same friction: looking up a manual whenever something broke, or digging for a laser cutter's specs just to design a case that would fit. He added an AI layer over the inventory. The director then posed the guiding question: could a student *describe a project* and have it **decomposed against the tools the lab actually has**? Pairing the inventory with each machine's manual and setup procedures turns a static list into something generative—a step-by-step account of the tooling and workflow a project requires. (An early version produced how-to infographics and renders; today the system generates grounded text that can seed an image on demand.)

The lab runs on lean staff and offers open access, and staff are often busy fabricating for classes and projects. When no one is on the floor, the knowledge of what each machine does, who may use it, and how to set it up is distributed, imperfect, and mostly in English—so access falls unevenly across students.

Three needs recur, and each routes today to a human bottleneck. Students need to **operate** a machine (where is it, how do I start, what training and PPE does it require?); to **debug** a machine and **log a maintenance ticket** when it throws an error mid-job; and to **scope** a project spanning several machines. The MakerLAB Assistant lowers all three at once—shared context for operating, planning, and creating in the lab.

2. System Overview

MakerLAB Tools treats the lab's operational knowledge as structured, typed data rather than a static list. The schema is normalized—tools, categories, locations, individual units, resources, setup/safety procedures, and maintenance logs—and hosted in Notion, keeping staff editing in a tool they already use. The web application (Next.js / React, deployed on Vercel) presents two surfaces: a **gallery** with fuzzy ranked search and category, material, training, and location facets across every published machine (Fig. 1), and a **tool detail** page showing description, location, required training, PPE and use restrictions, emergency-stop guidance, linked SOPs and manuals, and a live table of individual units with status, condition, and serial.

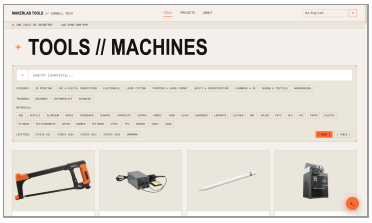

Fig. 1: The gallery—fuzzy search with category, training, material, and location facets across ~100 machines; the language selector sits top-right.

Overlaying both surfaces is the **MakerLAB Assistant**—a single unified assistant rather than a handful of separate tools (Fig. 2). From any page it answers questions, walks through setups, and logs maintenance tickets; near-term, it will also help create and manage inventory and schedule trainings. Rather than baking the catalog into a prompt, it reasons over the data through tool-calling, augmented with web search, document fetch (manuals and SOPs are pulled server-side and read in full), and vision (a photo of a machine or an error screen). Its context scales with place: from the gallery it carries a lightweight index of every tool and its resources; opened from a tool page it loads that machine's full detail and linked manuals, becoming a domain expert on the

machine in front of you. It answers in the language it is asked. The app is lightweight and **white-label**—a new lab connects its own Notion workspace through environment variables and hosts on serverless infrastructure—so other labs can adopt it.

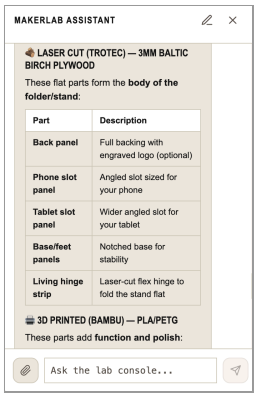

Fig. 2: Project scoping. The assistant decomposes a phone-and-tablet stand into laser-cut Baltic-birch parts (including a living hinge) and 3D-printed parts, each mapped to the right machine.

3. The Demonstration

The demonstration is interactive: visitors drive the live app.

3.1 Browse and scope.

Visitors browse the catalog, then ask the assistant to scope a project—e.g., *"I want to make a wooden stand for my phone and tablet with 3D-printed parts; which machines and steps would I need?"* It returns a start-to-finish, training-aware plan that decomposes the build into laser-cut plywood parts and 3D-printed parts, each mapped to the right machine and grounded in the lab's actual inventory (Fig. 2)—turning a vague idea into an executable plan.

3.2 Troubleshoot and log a ticket.

On a machine page, visitors ask a hands-on question—say, how to replace the filament on the Bambu Lab X1-Carbon. The assistant pulls the printer's SOP and walks through it step by step on the touchscreen (Fig. 3), in any language. And when a machine actually faults, the same chat files a maintenance ticket on the spot—so the machine stays online and the issue is captured rather than lost.

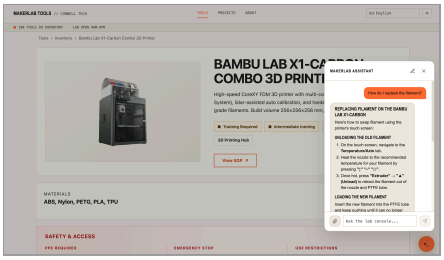

Fig. 3: A live exchange on the Bambu Lab X1-Carbon page—"how do I replace the filament?"—answered step by step from the printer's SOP (unload the old, load the new).

3.3 Bring your own AI (MCP).

The database is also a live, standards-based MCP server [1] (streamable HTTP) with tools to list, search, and inspect machines, units, and maintenance history. Attendees connect an LLM client—such as Claude [2] or ChatGPT [3]—and query the lab from a tool they already use—reaching the linked manuals and grounding a 3D print or laser-cut SVG in the lab's actual build and bed dimensions, coupling design files to the hardware that will make them.

3.4 Feedback and requirements.

We gather visitor feedback throughout. Requirements are light—the footprint of a standard table, plus power, reliable Wi-Fi, a couple of tablets, and a monitor.

4. Why a Demonstration

This work is best experienced, not read. The point is what the assistant takes off a student's plate—planning a build, recalling a setup, fixing a jam, filing a ticket—normally a hunt through manuals or a wait for staff. By simplifying these workflows, it lets students take on more ambitious projects, finish them faster, and learn the tools sooner—raising the number, complexity, and quality of what gets made. A demo is the best way to show this, gather feedback from students and other labs, and let those labs adopt it; the code will be openly accessible to read and run. As AI makes these labs easier to use and to teach, we expect more of them—and tools like this to help.

5. Early Deployment and Planned Evaluation

With the app live over the lab's real inventory, we are in a data-collection phase to assess whether it helps students. Early signals we will watch: whether more issues get surfaced and fixed (more uptime, less downtime), and whether students use the machines more because the lab is easier to navigate. We frame this as an in-progress study.

6. Discussion and Future Direction

This is an ongoing R&D initiative, not a finished product. Two near-term directions add the most value. First, a **student-projects gallery** linked to the devices that made each project—an archive and portfolio of student work and an institutional record of it. Second, **AI-assisted inventory creation** through the chat: a student uploads a few photos of tools or hardware and a short description, and the assistant fetches the manuals and drafts a catalog row for each—making inventory far faster to add and maintain. Farther out: a **digital twin** with a lab-wide agent that routes fabrication jobs and coordinates schedules and trainings; and, longest horizon, a shared operational framework across Cornell's campuses, so students move fluidly between labs while each keeps its identity.

Acknowledgements

We thank the staff and interns who helped build the MakerLAB; Cornell Tech and Cornell University; and all the students who have helped make it such an exciting, generative place. Isaac Steinberg designed the system architecture and wrote all application code for v5 and the current version, developed with the AI coding assistants Claude Code and Codex.

In accordance with ISAM policy, we disclose that a large language model (Anthropic's Claude) assisted in drafting and editing this abstract; all technical claims, design decisions, and the system itself are the authors' own.

References

[1] Anthropic, "Model Context Protocol," 2024. [Online]. Available: https://modelcontextprotocol.io. [Accessed: May 30, 2026].

[2] Anthropic, "Claude," 2023. [Online]. Available: https://www.anthropic.com/claude. [Accessed: May 30, 2026].

[3] OpenAI, "ChatGPT," 2022. [Online]. Available: https://openai.com/chatgpt. [Accessed: May 30, 2026].